Final Mini-Project Report

GPU Acceleration of Matrix Multiplication (GEMM)

Priyanshu Gonnade

Roll Number: 24B3959

January 26, 2026

1 Overview and Motivation

General Matrix Multiplication (GEMM) is a core computational kernel used extensively in scientific computing, signal processing, and deep learning workloads. The computational complexity of GEMM scales as $O(N^3)$, making naive CPU implementations prohibitively slow for large matrix sizes.

The objective of this project is to accelerate a real compute-heavy workload using GPU programming. A naive CPU implementation of GEMM is used as the baseline and compared against a hand-written CUDA kernel and a Triton-based GPU implementation. The focus is on performance improvement, correctness verification, and analysis of design trade-offs between CUDA and Triton.

2 CPU Baseline Implementation

2.1 Problem Definition

Given two square matrices **A** and **B** of size $N \times N$, the GEMM operation computes the output matrix **C** as:

$$C[i, j] = \sum_{k=0}^{N-1} A[i, k] \cdot B[k, j]$$

A naive triple-nested loop implementation was used on the CPU, without any parallelism or vectorization.

2.2 Experimental Setup

- Matrix size: $N = 512$
- Data type: 32-bit floating point
- Timing method: `time.perf_counter()`

2.3 CPU Results

The CPU baseline execution time for $N = 512$ was:

- **CPU GEMM Time: 37.10 seconds**

This clearly demonstrates the computational bottleneck and motivates GPU acceleration.

3 CUDA-Based GPU Implementation

3.1 Kernel Design

A naive CUDA GEMM kernel was implemented where each GPU thread computes one element of the output matrix. Threads are organized in a two-dimensional grid mapping directly to matrix rows and columns. Each thread performs a loop over the shared dimension k.

3.2 Timing Methodology

GPU execution time was measured using CUDA events to accurately capture kernel execution time while excluding host-device memory transfers. A warm-up kernel launch was performed prior to timing.

3.3 CUDA Results

Table 1: CUDA GEMM Performance

Matrix Size (N)	GPU Time (ms)
256	0.195
512	1.45
4096	575.3
8192	6852.3

For $N = 512$, the CUDA kernel achieved a speedup of approximately:

$$\frac{37.10}{0.00145} \approx 25{,}000\times$$

over the naive CPU implementation.

4 Triton-Based GPU Implementation

4.1 Motivation

Triton is a Python-first GPU programming framework that abstracts low-level thread management while enabling high-performance kernel execution. Triton was evaluated to compare productivity and performance against a hand-written CUDA kernel.

Since Triton does not support native Windows installations, experiments were conducted using a Linux-based Google Colab environment with GPU acceleration.

4.2 Benchmark Results

For $N = 512$, the Triton GEMM kernel achieved:

- **Triton GEMM Time: 0.619 ms**

This represents an additional speedup of approximately:

$$\frac{1.45}{0.619} \approx 2.3\times$$

over the naive CUDA implementation.

5 Correctness Verification

Correctness was verified prior to benchmarking.

5.1 CPU Reference Check

A naive CPU GEMM implementation was used as the reference. For deterministic inputs where all matrix elements were initialized to one, the analytically expected output value for each element is equal to N.

- Matrix size used for correctness: $N = 128$
- Maximum absolute error observed: 0.0

This confirms the correctness of the CPU and CUDA implementations.

5.2 Triton Verification

The Triton output was compared against PyTorch's `torch.matmul`. The maximum absolute error was below 10^{-5}, confirming numerical correctness.

6 Performance Analysis

The dramatic performance improvement from CPU to GPU arises from massive thread-level parallelism, high memory bandwidth, and efficient floating-point execution on GPUs.

For small matrix sizes, kernel launch overhead dominates execution time. As matrix size increases, the computation becomes compute-bound due to the $O(N^3)$ arithmetic intensity of GEMM.

7 CUDA vs Triton: Design Trade-offs

CUDA provides fine-grained control over hardware resources and enables highly optimized kernels but requires greater development effort. Triton significantly reduces code complexity and improves developer productivity while still delivering competitive performance.

8 Conclusion

This project demonstrates the successful acceleration of a real computational workload using GPUs. A naive CPU GEMM implementation taking tens of seconds was reduced to millisecond-scale execution using CUDA and Triton. Speedups exceeding four orders of magnitude were observed.

The results highlight the importance of GPU acceleration for compute-intensive workloads and show that Triton complements CUDA by offering a productive alternative for rapid development and research.